

claude-windows / ce5ec24b-cf40-49bb-afb0-3a41f4cc9934

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ce5ec24b-cf40-49bb-afb0-3a41f4cc9934\subagents\workflows\wf_2be29ecf-a31\agent-a2cce4a9fc2816b06.jsonl`
- SHA-256: `74e5964ba2541c1e220fcc189a52c2acddd264577dfcefab205875e5dbb255e9`
- Source modified: `2026-06-22T01:29:57+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_2be29ecf-a31`
- session_id: `ce5ec24b-cf40-49bb-afb0-3a41f4cc9934`
```

Transcript

1. user (2026-06-22T01:28:56.381Z)

Adversarially verify this P2b-install review finding. Read the cited code in C:\Users\avidu\Projects\bhi-p2b yourself (run `cd C:\Users\avidu\Projects\bhi-p2b && git diff origin/master...HEAD` and open the file). Decide if it is a REAL defect that should block or be fixed, or a false alarm / nit. Default to is_real=false if it does not hold up.

Lens: install-crossplatform

Severity(claimed): major

Title: Launcher runs bare `indexer` but neither install method guarantees it is on PATH

File: scripts/install.py

Detail: The launcher shortcuts emit a bare command ? Windows `.bat`: `indexer --app`; macOS `.command` / Linux `.sh`: `exec indexer --app` (scripts/install.py lines 73, 81, 89). This only works if the `indexer` console-script (pyproject `[project.scripts] indexer = "backend.main:main"`) is on PATH, but the two install methods do NOT guarantee that:

* `uv tool install` (install_command, line 126) installs into an isolated tool env and prints a warning when its bin dir (`~/.local/bin` on Unix, `%APPDATA%\uv\...` shim dir on Windows) is not on PATH ? it does NOT add it. The README change even asserts 'uv puts `indexer` on your PATH', which is false unless the user has run `uv tool update-shell` / `uv tool ensurepath`.

* `pip install` (line 127) uses `sys.executable -m pip`, dropping `indexer` into that interpreter's Scripts/bin dir, which on a system/global Python is frequently not on the interactive PATH.

Result: on a fresh machine the double-clicked shortcut can fail with 'indexer: command not found' / 'indexer' is not recognized', defeating the batteries-included goal. The shortcut content itself is correct (CRLF, quoted backslash path) ? the gap is PATH exposure. Suggested fix: After `uv tool install`, run `uv tool update-shell` (or detect and warn) so the tool-bin dir is on PATH. For both methods, make the shortcut robust to PATH gaps by either (a) resolving the absolute path to the `indexer` executable at install time (e.g. via the install-method's bin dir or `shutil.which('indexer')` after install) and embedding that absolute path in the shortcut, or (b) launching as `python -m backend.main --app` using the install-target interpreter. At minimum, print a loud post-install note when `shutil.which('indexer')` is None telling the user how to add it to PATH.

2. assistant (2026-06-22T01:28:59.786Z)

I'll verify this finding by examining the actual code in the cited repository.

3. user (2026-06-22T01:29:02.643Z)

README.md		31	++++++
backend/config/__init__.py		9	+-
backend/config/loader.py		34	++++++
backend/main.py		15	++++
docs/PACKAGING_PLAN.md		13	+-

```

scripts/install.py          | 210 ++++++
scripts/uninstall.py        | 150 ++++++
tests/test_app_launcher.py  | 42 ++++++
tests/test_p2b_install.py   | 163 ++++++
9 files changed, 660 insertions(+), 7 deletions(-)

```

4. assistant (2026-06-22T01:29:05.902Z)

Let me read the actual install.py file and the README change.

5. user (2026-06-22T01:29:06.647Z)

```

1      #!/usr/bin/env python3
2      """Batteries-included installer for the KhelSutra highlight indexer (PACKAGING_PLAN.md
P2b).
3
4      Cross-platform (Windows / macOS / Linux), no native component (01):
5
6      * installs the LIGHT (torch-free) package via ``uv tool install`` (preferred ? an
isolated tool
7      env that puts ``indexer`` on PATH) or ``pip install`` (fallback);
8      * creates a per-OS launcher shortcut that pins ``RALLY_APP_HOME`` to the OS app-data
dir and runs
9      ``indexer --app`` ? so config / .env / DB / reels live in ONE stable place
10     (mirrors ``backend.utils.app_home.default_os_app_home``). Pinning the env BEFORE the
process
11     starts is load-bearing: the engine's ``load_dotenv`` runs at import, so only an env
set by the
12     launcher (not by the engine) roots the ``.env`` in the app-home;
13     * writes an uninstall manifest so ``scripts/uninstall.py`` can cleanly reverse it.
14
15     The truly-local + ``motion`` default config is seeded by the ENGINE itself on the first
16     ``indexer --app`` run (``write_light_default_config``), so it works regardless of
install method;
17     this script only wires the install + the launcher.
18
19     Usage::
20
21     python scripts/install.py                # install '.' (this checkout), uv-
if-present else pip
22     python scripts/install.py --target badminton-highlight-indexer  # from PyPI (once
published)
23     python scripts/install.py --method pip    # force pip
24     python scripts/install.py --dry-run       # print the plan, change nothing
25
26     stdlib-only so it runs as a standalone bootstrap BEFORE the package exists.
27     """
28     from __future__ import annotations
29
30     import argparse
31     import json
32     import os
33     import platform
34     import shutil
35     import subprocess
36     import sys
37     from datetime import datetime, timezone
38     from pathlib import Path
39
40     PACKAGE = "badminton-highlight-indexer"
41     APP_DIR_NAME = "Khelsutra" # mirrors backend.utils.app_home.APP_DIR_NAME
42     MANIFEST_NAME = "uninstall_manifest.json"
43
44
45     # --- pure helpers (unit-tested)

```

```

-----
46
47     def default_os_app_home(app_name: str = APP_DIR_NAME, system: str | None = None,
48                             env: dict[str, str] | None = None) -> Path:
49         """The OS-conventional per-user app-data dir ? a standalone mirror of
50         ``backend.utils.app_home.default_os_app_home`` (this script runs before the package
51         exists)."""
52         system = system or platform.system()
53         e = os.environ if env is None else env
54         if system == "Windows":
55             base = e.get("APPDATA") or str(Path.home() / "AppData" / "Roaming")
56         elif system == "Darwin":
57             base = str(Path.home() / "Library" / "Application Support")
58         else:
59             base = e.get("XDG_DATA_HOME") or str(Path.home() / ".local" / "share")
60         return Path(base) / app_name
61
62     def shortcut_spec(system: str, app_home: Path, indexer_cmd: str = "indexer") ->
63     tuple[str, str, bool]:
64         """Return ``(filename, content, make_executable)`` for the per-OS launcher shortcut.
65         The shortcut pins ``RALLY_APP_HOME`` then runs ``indexer --app``. PURE (no IO) so
66         it's testable.
67         """
68         home = str(app_home)
69         if system == "Windows":
70             content = (
71                 "@echo off\r\n"
72                 "rem KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens
73                 the app.\r\n"
74                 f'set "RALLY_APP_HOME={home}"\r\n'
75                 f"{indexer_cmd} --app\r\n"
76             )
77             return "KhelSutra.bat", content, False
78         if system == "Darwin":
79             content = (
80                 "#!/bin/bash\n"
81                 "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
82                 app.\n"
83                 f'export RALLY_APP_HOME="{home}"\n'
84                 f"exec {indexer_cmd} --app\n"
85             )
86             return "KhelSutra.command", content, True
87         # Linux / other
88         content = (
89             "#!/bin/bash\n"
90             "# KhelSutra launcher (PACKAGING_PLAN.md P2b) ? pins app-home then opens the
91             app.\n"
92             f'export RALLY_APP_HOME="{home}"\n'
93             f"exec {indexer_cmd} --app\n"
94         )
95         return "khelsutra.sh", content, True
96
97     def build_manifest(*, method: str, app_home: Path, shortcut: Path, created: list[Path],
98                       timestamp: str) -> dict:
99         """The uninstall manifest (PURE). ``created`` = files THIS installer made (safe to
100         remove);
101         ``user_data`` = paths the engine/user own (kept unless ``uninstall.py --purge``)."""
102         return {
103             "schema": 1,
104             "product": "KhelSutra highlight indexer",
105             "package": PACKAGE,
106             "install_method": method,
107             # "uv" | "pip"

```

```

103         "indexer_command": "indexer",
104         "app_home": str(app_home),
105         "shortcut": str(shortcut),
106         "created": [str(p) for p in created],
107         "user_data": [
108             str(app_home / "config.json"),
109             str(app_home / ".env"),
110             str(app_home / "output"),
111         ],
112         "timestamp": timestamp,
113     }
114
115
116     def resolve_method(requested: str, uv_present: bool) -> str:
117         """Pick the installer: explicit beats auto; auto prefers uv when present, else
118         pip."""
119         if requested in ("uv", "pip"):
120             return requested
121         return "uv" if uv_present else "pip"
122
123     def install_command(method: str, target: str) -> list[str]:
124         """The argv for the chosen installer (PURE)."""
125         if method == "uv":
126             return ["uv", "tool", "install", target]
127         return [sys.executable, "-m", "pip", "install", target]
128
129
130     # --- IO / orchestration
131     -----
132     def _run(argv: list[str]) -> None:
133         print(f" $ {' '.join(argv)}")
134         subprocess.run(argv, check=True)
135
136
137     def main(argv: list[str] | None = None) -> int:
138         ap = argparse.ArgumentParser(description="Install the KhelSutra highlight indexer
139         (LIGHT tier).")
140         ap.add_argument("--target", default=".",
141             help="What to install: '.' (this checkout, default) or a PyPI
142             name/spec.")
143         ap.add_argument("--method", choices=("auto", "uv", "pip"), default="auto",
144             help="Installer to use (default: uv if present, else pip).")
145         ap.add_argument("--dry-run", action="store_true", help="Print the plan; change
146             nothing.")
147         args = ap.parse_args(argv)
148
149         system = platform.system()
150         app_home = default_os_app_home()
151         method = resolve_method(args.method, shutil.which("uv") is not None)
152         cmd = install_command(method, args.target)
153         fname, content, executable = shortcut_spec(system, app_home)
154         shortcut = app_home / fname
155
156         print(f"KhelSutra installer ? {PACKAGE} (LIGHT, torch-free)")
157         print(f" OS      : {system}")
158         print(f" installer : {method}")
159         print(f" target    : {args.target}")
160         print(f" app-home   : {app_home}")
161         print(f" shortcut   : {shortcut}")
162         if args.dry_run:
163             print("\n[dry-run] would run:")
164             print(f" $ {' '.join(cmd)}")
165             print(f" write shortcut -> {shortcut}")

```

```

163         print(f" write manifest -> {app_home / MANIFEST_NAME}")
164         print(f" copy uninstaller -> {app_home / 'uninstall.py'}")
165         return 0
166
167     print("\n[1/3] installing the package?")
168     try:
169         _run(cmd)
170     except (subprocess.CalledProcessError, FileNotFoundError) as e:
171         print(f"[ERROR] install failed: {e}")
172         if method == "uv":
173             print(" (uv not usable? re-run with --method pip)")
174         return 1
175
176     print("\n[2/3] writing the launcher shortcut + app-home?")
177     app_home.mkdir(parents=True, exist_ok=True)
178     shortcut.write_text(content, encoding="utf-8")
179     if executable:
180         shortcut.chmod(0o755)
181     # Make the uninstaller available next to the data it removes.
182     created: list[Path] = [shortcut]
183     here = Path(__file__).resolve().parent
184     src_uninstall = here / "uninstall.py"
185     dst_uninstall = app_home / "uninstall.py"
186     if src_uninstall.exists():
187         shutil.copyfile(src_uninstall, dst_uninstall)
188         created.append(dst_uninstall)
189
190     print("\n[3/3] writing the uninstall manifest?")
191     manifest_path = app_home / MANIFEST_NAME
192     created.append(manifest_path)
193     manifest = build_manifest(
194         method=method, app_home=app_home, shortcut=shortcut, created=created,
195         timestamp=datetime.now(timezone.utc).isoformat(timespec="seconds"),
196     )
197     manifest_path.write_text(json.dumps(manifest, indent=2) + "\n", encoding="utf-8")
198
199     print("\n? Installed. To launch KhelSutra:")
200     if system == "Windows":
201         print(f" double-click {shortcut} (or run `indexer --app`)")
202     else:
203         print(f" {shortcut} (or run `indexer --app`)")
204     print(" The first run seeds a truly-local 'motion' config (offline, no key/GPU
needed).")
205     print(f" To uninstall: python \"{dst_uninstall}\" (add --purge to also delete
your data)")
206     return 0
207
208
209 if __name__ == "__main__":
210     raise SystemExit(main())
211

```

6. assistant (2026-06-22T01:29:10.386Z)

The code matches the finding's citations exactly. Now let me check the README change to verify the "uv puts indexer on your PATH" claim.

7. user (2026-06-22T01:29:11.839Z)

```

diff --git a/README.md b/README.md
index 674e304..0bd46dd 100644
--- a/README.md
+++ b/README.md
@@ -32,6 +32,37 @@ The pipeline is built around three pluggable registries ? validators,
se

```

```

## ?? Setup & Diagnostics

#### 0. Batteries-included install (LIGHT tier ? recommended for end users)
+
+For a download-and-run experience (no `git clone`, no PyTorch, no GPU), use the cross-platform
+installer. It installs the **LIGHT tier** (torch-free: Gemini / `motion` / cpu-mock), wires a
+desktop launcher, and seeds a sensible default config.
+
+```bash
+# Preferred ? uv puts `indexer` on your PATH in an isolated tool env:
+uv tool install badminton-highlight-indexer          # (once published) or:  uv tool install .
+
+# Or the cross-platform installer script (uv-if-present, else pip) ? also creates a launcher
+# shortcut + an uninstall manifest:
+python scripts/install.py                          # add --method pip to force pip, --dry-run
to preview
+```
+
+Then launch the app (the installer's shortcut, or directly):
+
+```bash
+indexer --app          # boots a local server (127.0.0.1) and opens the bundled KhelSutra UI
+```
+
+* **First run seeds a truly-local default** ? detector `motion` (offline, **no API key, no GPU,
+ no cloud bills**); the in-app setup later switches you to Gemini. State (config, DB, reels)
+lives
+ in your OS app-data dir (`%APPDATA%\Khelsutra` / `~/Library/Application Support/Khelsutra` /
+ `~/.local/share/Khelsutra`) when launched via the installer's shortcut.
+* **FFmpeg is still required** (see step 1) ? it is intentionally not redistributed (GPL
+`libx264`).
+* **Uninstall cleanly:** `python uninstall.py` (dropped next to your data) ? keeps your
+config/reels
+ by default; add `--purge` to delete them too.
+
+The native-GPU (WASB) and cloud tiers are separate, opt-in provisioning ? see
+`docs/PACKAGING_PLAN.md`.
+
+### 1. External System Requirements
+The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes,
+and stitch clips.

```

8. assistant (2026-06-22T01:29:15.489Z)

The README does claim "uv puts `indexer` on your PATH in an isolated tool env". Now let me verify the factual claim about `uv tool install` behavior ? does it actually add the bin dir to PATH, or only warn? Let me check what version of uv is available and verify the behavior, plus check the pyproject for the console script.

9. user (2026-06-22T01:29:16.331Z)

```

84:[project.scripts]
85-# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
86-indexer = "backend.main:main"
87-
88-uv version---
89-uv not installed

```

10. assistant (2026-06-22T01:29:21.839Z)

The console script `indexer = "backend.main:main"` is confirmed. uv is not installed locally so I can't run it, but the `uv tool install` PATH behavior is well-documented and stable: it installs into `~/.local/bin` (Unix) / `%APPDATA%\uv\tools\...` shim dir on Windows, and when that directory isn't on PATH it prints a warning ("...is not on your PATH. To use installed

tools, run `export PATH=...` or `uv tool update-shell`) but does NOT modify PATH automatically. This is accurate as of all current uv versions.

Let me verify the test file to see whether the install path scenario is tested, and check whether there's any post-install PATH check that the finding may have missed.

11. user (2026-06-22T01:29:22.091Z)

```
1      """P2b-install (PACKAGING_PLAN.md): batteries-included config seeding +
install/uninstall tooling.
2
3      Covers:
4      * the engine-side ``write_light_default_config`` (gate 08 default = truly-local +
motion), incl.
5      the load-bearing **no-overwrite** idempotency and the EFFECTIVE config the loader
produces;
6      * the pure logic of ``scripts/install.py`` / ``scripts/uninstall.py`` (shortcut
content, manifest,
7      installer/uninstaller argv, method resolution, OS app-home) ? no real subprocess/IO.
8      """
9      from __future__ import annotations
10
11     import importlib.util
12     import json
13     from pathlib import Path
14
15     import pytest
16
17     from backend.config import LIGHT_DEFAULT_CONFIG, load_config, write_light_default_config
18
19
20     def _load_script(name: str):
21         """Load a scripts/*.py bootstrap file as a module (it is stdlib-only, safe to
exec)."""
22         path = Path(__file__).resolve().parents[1] / "scripts" / name
23         spec = importlib.util.spec_from_file_location(name[:-3], path)
24         assert spec and spec.loader
25         mod = importlib.util.module_from_spec(spec)
26         spec.loader.exec_module(mod)
27         return mod
28
29
30     # --- engine seeding: write_light_default_config (gate 08)
-----
31
32     def test_seed_writes_truly_local_motion(tmp_path):
33         cfg = tmp_path / "sub" / "config.json" # also proves parent-dir creation
34         path, written = write_light_default_config(cfg)
35         assert written is True and path == cfg and cfg.exists()
36         data = json.loads(cfg.read_text(encoding="utf-8"))
37         assert data == LIGHT_DEFAULT_CONFIG
38         assert data["indexing"]["default_segmenter"] == "motion"
39         assert data["indexing"]["use_gpu"] is False
40         assert data["deployment"]["profile"] == "truly-local"
41
42
43     def test_seed_does_not_overwrite_existing(tmp_path):
44         cfg = tmp_path / "config.json"
45         cfg.write_text('{"sport": "tennis", "indexing": {"default_segmenter": "gemini"}}',
encoding="utf-8")
46         path, written = write_light_default_config(cfg)
47         assert written is False and path == cfg
48         # The user's file is preserved untouched (the wizard/edits are never clobbered).
49         assert json.loads(cfg.read_text(encoding="utf-8"))["indexing"]["default_segmenter"]
== "gemini"
```

```

50
51
52 def test_seed_overwrite_true_forces(tmp_path):
53     cfg = tmp_path / "config.json"
54     cfg.write_text("{} ", encoding="utf-8")
55     _, written = write_light_default_config(cfg, overwrite=True)
56     assert written is True
57     assert json.loads(cfg.read_text(encoding="utf-8")) == LIGHT_DEFAULT_CONFIG
58
59
60 def test_seeded_config_loads_to_truly_local_motion(tmp_path):
61     """The minimal seeded file, run through the loader, expands via the truly-local
PRESET to a
62     coherent torch-free/offline config (motion + no Gemini handoff + local storage + no
GPU)."""
63     cfg = tmp_path / "config.json"
64     write_light_default_config(cfg)
65     loaded = load_config(cfg)
66     assert loaded.indexing.default_segmenter == "motion"
67     assert loaded.indexing.use_gpu is False
68     assert loaded.deployment.profile == "truly-local"
69     assert loaded.indexing.skip_ai_handoff is True          # supplied by the truly-local
preset
70     assert loaded.storage.backend == "local"                # supplied by the truly-local
preset
71
72
73     # --- scripts/install.py pure logic
-----
74
75     @pytest.fixture(scope="module")
76     def install_mod():
77         return _load_script("install.py")
78
79
80     @pytest.fixture(scope="module")
81     def uninstall_mod():
82         return _load_script("uninstall.py")
83
84
85     @pytest.mark.parametrize(
86         "system, fname, executable",
87         [("Windows", "KhelSutra.bat", False), ("Darwin", "KhelSutra.command", True),
88         ("Linux", "khelsutra.sh", True)],
89     )
90     def test_shortcut_spec_per_os(install_mod, system, fname, executable):
91         home = Path("/home/u/AppHome")
92         name, content, ex = install_mod.shortcut_spec(system, home)
93         assert name == fname and ex is executable
94         # str(home) is OS-native (backslashes on Windows) ? assert against it, not a
hardcoded literal.
95         assert "RALLY_APP_HOME" in content and str(home) in content
96         assert "indexer --app" in content
97
98
99     def test_resolve_method(install_mod):
100         assert install_mod.resolve_method("auto", uv_present=True) == "uv"
101         assert install_mod.resolve_method("auto", uv_present=False) == "pip"
102         assert install_mod.resolve_method("pip", uv_present=True) == "pip"    # explicit wins
103         assert install_mod.resolve_method("uv", uv_present=False) == "uv"
104
105
106     def test_install_command_argv(install_mod):
107         assert install_mod.install_command("uv", ".") == ["uv", "tool", "install", "."]
108         pip = install_mod.install_command("pip", "badminton-highlight-indexer")

```



```

109         assert pip[1:] == ["-m", "pip", "install", "badminton-highlight-indexer"]
110
111
112     def test_default_os_app_home_per_os(install_mod):
113         win = install_mod.default_os_app_home(system="Windows", env={"APPDATA":
114 r"C:\Users\u\AppData\Roaming"})
115         assert win.name == "Khelsutra" and "Roaming" in str(win)
116         lin = install_mod.default_os_app_home(system="Linux", env={"XDG_DATA_HOME":
117 "/home/u/.local/share"})
118         assert str(lin) == str(Path("/home/u/.local/share") / "Khelsutra")
119
120
121     def test_build_manifest_shape(install_mod):
122         home = Path("/home/u/AppHome")
123         m = install_mod.build_manifest(
124             method="uv", app_home=home, shortcut=home / "khelsutra.sh",
125             created=[home / "khelsutra.sh", home / "uninstall_manifest.json"],
126             timestamp="2026-06-22T00:00:00+00:00",
127         )
128         assert m["package"] == "badminton-highlight-indexer"
129         assert m["install_method"] == "uv"
130         assert str(home / "config.json") in m["user_data"] # user data is tracked
131         separately (kept on uninstall)
132         assert m["shortcut"] == str(home / "khelsutra.sh")
133         assert m["schema"] == 1
134
135     # --- scripts/uninstall.py pure logic
136     -----
137
138     def test_uninstall_command_argv(uninstall_mod):
139         assert uninstall_mod.uninstall_command("uv", "badminton-highlight-indexer") == \
140             ["uv", "tool", "uninstall", "badminton-highlight-indexer"]
141         pip = uninstall_mod.uninstall_command("pip", "badminton-highlight-indexer")
142         assert pip[1:] == ["-m", "pip", "uninstall", "-y", "badminton-highlight-indexer"]
143
144
145     def test_find_manifest_explicit(uninstall_mod, tmp_path):
146         m = tmp_path / "uninstall_manifest.json"
147         m.write_text("{} ", encoding="utf-8")
148         assert uninstall_mod.find_manifest(str(m)) == m
149         assert uninstall_mod.find_manifest(str(tmp_path / "nope.json")) is None
150
151
152     def test_install_uninstall_roundtrip_manifest(install_mod, uninstall_mod, tmp_path):
153         """The manifest install.py would write is consumable by uninstall.py's
154         expectations."""
155         home = tmp_path / "AppHome"
156         m = install_mod.build_manifest(
157             method="pip", app_home=home, shortcut=home / "KhelSutra.bat",
158             created=[home / "KhelSutra.bat", home / "uninstall_manifest.json"],
159             timestamp="2026-06-22T00:00:00+00:00",
160         )
161         mp = home / "uninstall_manifest.json"
162         home.mkdir(parents=True)
163         mp.write_text(json.dumps(m), encoding="utf-8")
164         assert uninstall_mod.find_manifest(str(mp)) == mp
165         loaded = json.loads(mp.read_text(encoding="utf-8"))
166         assert uninstall_mod.uninstall_command(loaded["install_method"],
167             loaded["package"])[0:3] in (
168             ["uv", "tool", "uninstall"], [install_mod.sys.executable, "-m", "pip"],
169         )
170

```

I now have the full picture. Let me confirm there's no post-install PATH check anywhere in `install.py` (the finding says there is none ? ``main()`` ends right after writing the manifest with no ``shutil.which('indexer')`` check).

Confirmed: ``main()`` (lines 137-206) has no ``shutil.which('indexer')`` check, no ``uv tool update-shell``, no PATH warning. The post-install message at lines 199-205 unconditionally tells the user to double-click the shortcut or run ``indexer --app``.

Let me assess the factual accuracy of each sub-claim:

1. ****Launcher emits bare ``indexer --app`` ?** Confirmed at lines 73, 81, 89 (via ``indexer_cmd="indexer"`` default). True.
2. ****``uv tool install`` does not add bin dir to PATH, only warns**** ? This is accurate and well-documented uv behavior. ``uv tool install`` places executables in the user tool-bin dir and prints a warning when it's not on PATH, requiring ``uv tool update-shell`` / ``uv tool ensurepath`` to actually modify PATH. It does not auto-modify PATH. True.
3. ****README asserts "uv puts ``indexer`` on your PATH"***** ? Confirmed in the README diff: "uv puts ``indexer`` on your PATH in an isolated tool env". This is misleading/false in the fresh-machine case. True.
4. ****``pip install`` uses ``sys.executable -m pip``, dropping ``indexer`` into that interpreter's Scripts/bin which is frequently not on interactive PATH**** ? Confirmed at line 127. On Windows especially, ``python -m pip install`` into a system/store Python places the console script in a ``Scripts`` dir that is often not on PATH; with ``--user`` installs the situation is even worse. True.
5. ****No post-install PATH check / fallback**** ? Confirmed: no ``shutil.which`` check exists in the orchestration. True.

Now, the verification question: is this a REAL defect that should block or be fixed, or a false alarm/nit?

This is a genuine functional gap for a "batteries-included, double-click-to-run" installer whose explicit O1 goal (per the module docstring) is a download-and-run experience for end users. The most-likely failure mode ? a non-technical user on a fresh machine double-clicks ``KheISutra.bat`` and gets ``'indexer' is not recognized`` ? directly defeats the stated goal. The README also contains a factually incorrect PATH claim. Both the install scripts and the prose are in scope for this PR (P2b-install).

It is not a blocker in the strict sense (the package does install; a technically-savvy user can run ``uv tool update-shell`` or invoke the absolute path), and there is a documented escape hatch (``indexer --app`` from an activated env). But the core promise of the shortcut is unreliable on common configurations. This lands as a ****major**** install-cross-platform issue: the headline UX path can fail silently for the target persona, and the README makes an inaccurate guarantee.

The suggested fixes are sound (embed absolute path via ``shutil.which``/the tool-bin dir, or ``python -m backend.main --app`` with the install-target interpreter, or at minimum run ``uv tool update-shell`` + a loud PATH warning).

13. user (2026-06-22T01:29:52.029Z)

Structured output provided successfully

14. assistant (2026-06-22T01:29:56.924Z)

Verified the finding against the actual code in `C:\Users\avidu\Projects\bhi-p2b`. It holds up as a real major issue: the launcher shortcuts run a bare ``indexer --app``, but neither ``uv tool install`` nor ``pip install (sys.executable -m pip)`` guarantees that console script is on PATH, and there's no post-install PATH check/fallback. The README also makes an inaccurate "uv puts ``indexer`` on your PATH" claim. Details recorded via StructuredOutput.